

Trabajo Práctico de Programación C
Programación en Lenguaje C

Ejercicios

1. Considerar los siguientes planteos en el contexto del lenguaje C:
 - a) ¿Qué diferencia una estructura de datos estática de una dinámica? Encontrar al menos dos ejemplos de cada categoría.
 - b) ¿Una variable de tipo puntero en sí misma, constituye una estructura de datos estática o dinámica?
 - c) ¿Cuál es la utilidad de las funciones de librería `malloc()` y `free()`?
 - d) ¿Cómo hace el compilador para saber cuánta memoria debe reservar en el **HEAP** ante cada invocación a la función `malloc()`?
 - e) Analice las diferencias que presentan los datos almacenados en **STACK**, frente aquellos que son almacenados en **HEAP**. Tenga en cuenta las siguientes características:
 - Alcance de los datos.
 - Persistencia de los datos: temporal, permanente.
2. Suponer un programa que, haciendo uso de la función `tri`, triplica el valor de un entero. De las soluciones planteadas a continuación todas compilan, sin embargo, no todas las soluciones son correctas (aunque algunas funcionen). Analice cada una de ellas e identifique cuál es el problema.

a)

<pre>void main(){ int a = 34; int a3; int* p; p = tri(a); a3 = *p; free(p); }</pre>	<pre>int* tri(int x){ int* res; res = (int*) malloc(sizeof(int)); *res = 3 * x; return res; }</pre>
---	---

b)

<pre>void main(){ int a = 34; int a3; a3 = tri(a); }</pre>	<pre>int tri(int x){ int* res; res = (int*) malloc(sizeof(int)); *res = 3 * x; return *res; }</pre>
--	---

c)

<pre>void main(){ int* a; a = (int*)malloc(sizeof(int)); *a = 34; tri(a); }</pre>	<pre>void tri(int* x){ int res; res = *x * 3; *x = res; }</pre>
---	---

3. Escriba declaraciones de tipos adecuadas para representar la siguiente información:

a) *Información sobre un empleado:*

- apellido (máximo 40 caracteres)
- nombre (máximo 40 caracteres)
- domicilio: calle, número y especificaciones (torre, piso, dpto.)
- teléfono
- sexo (F,M)
- edad (entre 18 y 65 años)
- estudios cursados (primarios, secundarios, terciarios, universitarios, posgrado)
- estado civil (soltero, casado, viudo, divorciado)

b) *Información sobre automóviles*

- marca (máximo 20 caracteres)
- nombre del modelo (ejemplo: symbol, megane)
- modelo (año)
- patente (3 letras y 3 números)
- estado (excelente, muy bueno, bueno, regular, malo)
- precio

Encapsule cada definición en un archivo '.h' diferente (*empleado.h* y *automovil.h*). Analice qué funciones debería implementar para permitir el ingreso y modificación de la información de ambos tipos, especificando en qué archivos debería incluir la implementación de las mismas y por qué.

4. Dadas dos variables *empleado_1* y *empleado_2* del tipo *struct empleado* definido en el ejercicio anterior, indique qué sucede al efectuar la siguiente sentencia:

empleado_1 = empleado_2;

5. Considerando la definición del **struct usuario** dada a continuación, suponga un programa que haciendo uso de una función **nuevo_usuario**, inicializa y retorna un nuevo *usuario*. De las soluciones planteadas en cada una de las funciones a continuación, todas compilan, sin embargo, no todas las soluciones son correctas (aunque algunas funcionen). Analice cada una de ellas e identifique cuál es el problema.

```
typedef struct usuario{
    char apellido [41];
    char nombre [41];
} * tUsuario;
```

//CONTINÚA EN LA SIGUIENTE PÁGINA

```

struct usuario nuevo_usuario_1(){
    char n [41], a [41];
    struct usuario u;
    printf("Ingrese nombre: "); scanf("%40[^\n]", n); fflush(stdin);
    printf("Ingrese apellido: "); scanf("%40[^\n]", a); fflush(stdin);
    strcpy(u.nombre, n);
    strcpy(u.apellido, a);
    return u;
}

```

```

tUsuario nuevo_usuario_2(){
    char n [41], a [41];
    tUsuario u;
    printf("Ingrese nombre: "); scanf("%40[^\n]", n); fflush(stdin);
    printf("Ingrese apellido: "); scanf("%40[^\n]", a); fflush(stdin);
    strcpy(u->nombre, n);
    strcpy(u->apellido, a);
    return u;
}

```

```

tUsuario nuevo_usuario_3(){
    char n [41], a [41];
    tUsuario u = (tUsuario) malloc(sizeof(struct usuario));
    printf("Ingrese nombre: "); scanf("%40[^\n]", n); fflush(stdin);
    printf("Ingrese apellido: "); scanf("%40[^\n]", a); fflush(stdin);
    strcpy(u->nombre, n);
    strcpy(u->apellido, a);
    return u;
}

```

```

tUsuario nuevo_usuario_4(){
    char n [41], a [41];
    struct usuario u;
    printf("Ingrese nombre: "); scanf("%40[^\n]", n); fflush(stdin);
    printf("Ingrese apellido: "); scanf("%40[^\n]", a); fflush(stdin);
    strcpy(u.nombre, n);
    strcpy(u.apellido, a);
    return &u;
}

```

//CONTINÚA EN LA SIGUIENTE PÁGINA

```

int main(){
    int i;
    tUsuario u, users [4];
    struct usuario u_1;

    u_1 = nuevo_usuario_1();
    users[0] = &u_1;
    users[1] = nuevo_usuario_2();
    users[2] = nuevo_usuario_3();
    users[3] = nuevo_usuario_4();

    for(i=0; i<4; i++){
        u = users[i];
        printf("Usuario %i: \n", i+1);
        printf("Nombre: %s \n", u->nombre);
        printf("Apellido: %s \n", u->apellido);
    }
    return 0;
}

```

6. Codificar una función que lea los números de libreta (enteros) y las calificaciones (caracteres) de los alumnos de un curso, almacenando la información suministrada en dos arreglos unidimensionales, para luego retornar la cantidad de alumnos ingresados. Finalmente, escribir un programa de prueba que invoque a la función anterior y luego muestre por pantalla la información ingresada.
7. Modificar el programa anterior para que haciendo uso de registros utilice un único arreglo unidimensional.

Referencias

- [KR88] KERNIGHAN, B., AND RITCHIE, D. *The C Programming Language*, second ed. Prentice-Hall, 1988.
- [Wik] *C Programming*. https://en.wikibooks.org/wiki/C_Programming
 Pdf: https://upload.wikimedia.org/wikipedia/commons/0/07/C_Programming.pdf.